

XIII Maratona de Programação UnB

Caderno de Problemas



13^a MARATONA UNB DE PROGRAMAÇÃO

3 de novembro de 2025

(Este caderno contém 14 problemas)

Organização



Comissão Organizadora

Arthur Botelho (UnB)

Daniel Porto (UnB)

Edson A. C. Júnior (UnB/FCTE)

Eduardo Freire (UnB)

Eduardo Quirino (UnB)

Emerson Cruz (UnB)

Guilherme N. Ramos (UnB)

Guilherme Rocha (UnB).

Jeremias Gomes (IDP).

Lucas Gabriel Lima (UnB)

Maxwell Oliveira (UnB)

Pedro Avila Beneveli (UnB)

Ruan Petrus (UnB)

Wilson Guimarães (UnB)

Lembretes

- É permitido consultar livros, anotações ou qualquer outro material impresso durante a prova, entretanto, o mesmo não vale para materiais dispostos eletronicamente.
- A correção é automatizada, portanto, siga atentamente as exigências da tarefa quanto ao formato da entrada e saída conforme as amostras dos exemplos. Deve-se considerar entradas e saídas padrão;
- Para cada problema, além dos testes públicos, o juiz executará a sua submissão contra uma série de testes secretos para fornecer um parecer sobre a correção do programa.
- Procure resolver o problema de maneira eficiente. Se o tempo superar o limite pré-definido, a solução não é aceita. Lembre-se que as soluções são testadas com outras entradas além das apresentadas como exemplo dos problemas;
- Utilize a aba *clarification* para dúvidas da prova. Os juízes podem opcionalmente atendê-lo com respostas acessíveis a todos;

C/C++

- Seu programa deve retornar zero, executando, como último comando, `return 0` ou `exit 0`.

Java

- Não declare ‘`package`’ no seu programa Java.
- Note que a convenção para o nome do arquivo fonte deve ser obedecida, o que significa que o nome de sua classe pública deve ser uma letra maiúscula igual a letra que identifica o problema.

Python

- Tenha cuidado ao selecionar a versão correta na submissão.

Problem A. Análise de Corridas

Input file: **standard input**
Output file: **standard output**
Time limit: 2.5 seconds
Memory limit: 256 megabytes

Maxwell, um grande entusiasta de todas as formas de corrida, decidiu organizar o Campeonato Mundial de Kart da UnBalloon! Nesse campeonato, foram escolhidos os melhores pilotos do grupo, cada um representando a inicial de seu nome, ou seja, foram escolhidos 26 competidores, um para cada letra do alfabeto. Estes competidores terão que competir por n voltas emocionantes, disputando o lendário título de **felipe_massa**.

A grande jornalista e fotógrafa Isa decidiu fazer uma matéria sobre esse evento. Porém, envolvida por toda a disputa e emoção da corrida, ela conseguiu gravar apenas os vencedores de m voltas consecutivas! Isa não consegue se recordar em que momento ela começou a fazer essas anotações, porém tem certeza de que elas estão corretas e ocorreram em algum momento da corrida e que cada letra representa qual piloto ganhou aquela volta. Para conseguir terminar sua matéria, ela precisa saber de quantas formas a corrida como um todo pode ter ocorrido seguindo suas anotações, sabendo que uma corrida é caracterizada apenas pelos vencedores de cada volta.

Com isso, cabe a você, um aspirante ao título de **felipe_massa** e amante de corrida, responder essa pergunta. Como o número de corridas distintas pode ser muito grande, Isa pediu que esse cálculo seja realizado módulo 998244353. Duas corridas são consideradas diferentes se, em alguma volta i , o vencedor dessa volta i é diferente entre as corridas.

Input

A primeira linha da entrada contém dois inteiros n e m ($1 \leq n \leq 10^{18}$, $1 \leq m \leq 100$, $m \leq n$), o número total de voltas e o tamanho das anotações de Isa.

A segunda linha contém uma string s , as anotações da corrida, composta por m caracteres minúsculos do alfabeto latino.

Output

Imprima a quantidade de corridas que podem ter ocorrido de acordo com as anotações de Isa, módulo 998244353.

Examples

standard input	standard output
6 5 ilprw	52
8 5 ababa	70252
100000000 15 novofelipemassa	966021064

Note

No primeiro caso de teste, Isa anotou o resultado de 5 voltas, com os vencedores de cada volta sendo “ilprw”. Como a corrida teve um total de 6 voltas, podem ter acontecido as corridas “Xilprw” ou “ilprwX”, em que X representa qualquer um dos 26 competidores.

Problem B. Bauru

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

Em Bauru, a numeração dos prédios é bastante diferenciada. Em vez de o endereço ser apenas Rua r Número Y , é da forma: Rua r Quadra Y Número Z . Para facilitar a escrita, o endereço é normalmente reduzido para: Rua r , Número $Y - Z$, ou seja, unindo o número da quadra com o do prédio. Por exemplo: Rua Darcy Ribeiro, 9 – 75 (quadra 9, número 75).

Dhara mora em Bauru, e uma de suas amigas lhe enviou uma mensagem pedindo que enviasse uma encomenda para a Rua Quintino, $A - B$ (quadra A , número B). Porém, em vez de separar $A - B$, a mensagem apresentava os dois números juntos, da forma AB !

Ajude Dhara a separar a mensagem AB em A e B , para que ela possa enviar a encomenda. Para isso, leve em consideração que A e B são números entre 1 e 99 e que não foram escritos com zeros à esquerda (por exemplo: se $A = 1$, não pode ter sido escrito como 01 ou 0001, apenas 1).

Input

A entrada contém apenas uma linha, com a mensagem AB . AB é uma string numérica que tem de 2 a 4 dígitos.

É garantido que AB pode ser formada a partir de números A e B válidos, de acordo com o enunciado.

Output

Se for possível separar AB em A e B de forma única, imprima A e B separados por um espaço, em uma única linha.

Caso contrário, ou seja, se existem múltiplos A e B válidos que formem AB , imprima -1 .

Examples

standard input	standard output
975	-1
11	1 1
1010	10 10
120	1 20

Problem C. Cada K

Input file: **standard input**
Output file: **standard output**
Time limit: **1 second**
Memory limit: **256 megabytes**

Adrielly gosta bastante de música, e para tornar suas idas e vindas de ônibus da UnB menos maçantes, decidiu fazer a sua própria playlist!

Para isso, ela separou as N últimas músicas que ela escutou (numeradas de 1 a N) e, para cada música i , atribuiu um valor A_i . Esse valor representa o quanto Adrielly gosta dessa música — se for negativo, é uma música que ela se arrependeu bastante de ter escutado...

Obviamente, Adrielly gostaria de fazer uma playlist que maximize a soma dos valores das músicas escolhidas! No entanto, atualmente ela está com uma música na cabeça com o refrão: “Um a Cada K ! Um a Cada K !...”. Por isso, se ela escolher a música i para entrar na playlist, ela não poderá escolher nenhuma música j com $i < j < i + K$.

Ajude Adrielly a tornar as suas idas e vindas mais legais dizendo a maior soma possível dos valores de uma playlist, obedecendo à restrição de “Um a Cada K ”!

Input

A primeira linha da entrada contém dois números: N e K ($1 \leq K \leq N \leq 10^5$). É garantido que $K \cdot N \leq 10^5$.

A segunda linha da entrada contém os N valores A_i ($-10^9 \leq A_i \leq 10^9$).

Output

Imprima uma única linha contendo a maior soma possível dos valores de uma playlist, respeitando a restrição do enunciado.

Examples

standard input	standard output
12 3 1 4 -5 6 19 20 -1 4 5 -3 0 10	39
3 1 1 2 3	6
3 1 -1 -2 -3	0
4 4 2 3 0 2	3

Note

No primeiro caso de exemplo, a melhor opção para Adrielly é escolher as músicas com os valores: 4, 20, 5, e 10.

No segundo caso de exemplo, Adrielly consegue a soma máxima escolhendo todas as músicas.

No terceiro caso de exemplo, é melhor Adrielly não escolher nenhuma música, porque todas são ruins... Então a melhor soma é feita com uma playlist vazia (soma 0).

No quarto caso de exemplo, Adrielly deve escolher a música de valor 3.

Problem D. Djsb ef Tijgu

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

Arthur é um espião da renomada agência secreta MI-9548. Em suas missões, por questões de segurança, ele usa o mais sofisticado algoritmo de criptografia para se comunicar com o quartel general: a Cifra de Shift.

As mensagens enviadas por Arthur são representadas por listas contendo valores inteiros entre 0 a 9999. A Cifra de Shift consiste em escolher um “shift” entre 0 e 9999 e adicionar esse mesmo valor à todos os elementos da lista. Caso essa aplicação faça o valor do elemento ultrapassar 9999, deve-se tomar o resto do valor na divisão por 10000.

Por exemplo, se a mensagem de Arthur for [1, 2, 3, 4] e escolhermos o shift 3, teremos a nova lista [4, 5, 6, 7]. Se escolhermos o shift 9998, teremos a lista [9999, 0, 1, 2].

Arthur tem uma frase favorita B , representada por uma lista de números, e gosta escolher o shift de criptografia de modo que essa frase apareça o maior número possível de vezes na mensagem criptografada.

Por exemplo, se a mensagem a ser criptografada é [1, 2, 3, 1, 2] e Arthur gosta da frase [6, 7], ele deve escolher o shift 5, já que isso resultará na mensagem [6, 7, 8, 6, 7], que contém a frase de Arthur duas vezes.

Ajude Arthur, encontrando qual é o shift que resulta no maior número de ocorrências da frase favorita dele na mensagem A a ser criptografada, e responda adicionalmente qual é o número de ocorrências. Se mais de um shift maximiza o número de ocorrências, encontre o menor deles.

Input

A primeira linha da entrada contém dois inteiros N e M ($2 \leq M \leq N \leq 10^6$), que correspondem ao comprimento da mensagem A e o comprimento da frase B respectivamente.

A segunda linha contém N inteiros separados por espaço: a mensagem A ($0 \leq A_i < 10^4$).

A terceira linha contém M inteiros separados por espaço: a frase B ($0 \leq B_i < 10^4$).

Output

A saída deve conter dois números separados por um espaço: um valor entre 0 e 9999, representando o menor shift que maximiza as ocorrências da frase, e o número de ocorrências.

Examples

standard input	standard output
5 2 1 2 3 1 2 6 7	5 2
4 2 1 2 3 4 0 0	0 0
2 2 452 453 0 1	9548 1

Problem E. Estrutura de Dados Mais Fraca

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

Decidida a aprimorar suas habilidades, Nikolle escalou o monte mais alto conhecido pela humanidade, indo além das nuvens. Lá, ela encontrou um monastério, onde vivia o grande mestre Z. Treinando com ele, Nikolle adquiriu conhecimentos poderosos sobre programação competitiva, como Segment Tree Beats, Sweepline Mo e 3D Hull Online.

Depois de meses de aprimoramento, Nikolle voltou para a UnB e decidiu compartilhar o seu conhecimento com seus colegas. Porém, antes de chegar em assuntos complexos demais, ela decidiu propor um desafio mais simples: implementar a estrutura de dados mais fraca que ela conhece, a Teg Serpistente.

Uma Teg Serpistente mantém uma lista de pilhas, inicialmente sem nenhuma pilha. A estrutura suporta as seguintes operações:

- Tipo 1: adicionar uma pilha vazia ao final da lista. Se a lista tinha S pilhas antes dessa adição, a pilha adicionada recebe o identificador $S + 1$.
- Tipo 2: receber i e x , e adicionar o número x ao topo da pilha i .
- Tipo 3: receber i e j , e imprimir o j -ésimo número adicionado à pilha i (se a pilha tem T números, o elemento do topo foi o T -ésimo adicionado).

Mostre para Nikolle que você tem o que é necessário para aprender suas técnicas!

Input

A primeira linha da entrada contém um inteiro Q : a quantidade de operações a serem realizadas ($1 \leq Q \leq 100$).

Cada uma das próximas Q linhas descreve uma operação a ser realizada, e as operações devem ser realizadas na ordem em que são dadas na entrada. Essas linhas começam com um inteiro t , indicando o tipo da operação de acordo com o enunciado ($1 \leq t \leq 3$).

- Se $t = 1$, não há outros inteiros nesta linha.
- Se $t = 2$, seguem dois inteiros i e x nesta linha. É garantido que, antes de realizar esta operação, a lista tenha pelo menos i pilhas e que $1 \leq x \leq 100$.
- Se $t = 3$, seguem dois inteiros i e j nesta linha. É garantido que, antes de realizar esta operação, a lista tenha pelo menos i pilhas e que a pilha i tenha pelo menos j números.

Output

Para toda operação do tipo 3, imprima o j -ésimo número adicionado à pilha i . Imprima apenas um número por linha, na ordem em que aparecem na entrada.

Example

standard input	standard output
13	3
1	5
2 1 3	4
3 1 1	2
1	3
2 1 4	
2 2 5	
3 2 1	
3 1 2	
1	
2 3 1	
2 3 2	
3 3 2	
3 1 1	

Problem F. Falatro

Input file: standard input
Output file: standard output
Time limit: 2 seconds
Memory limit: 256 megabytes

Falatro é um jogo de cartas que tem se popularizado muito nos últimos tempos.



Ele é jogado com um baralho quase comum e o objetivo é formar uma sequência de cartas com a maior pontuação possível, seguindo algumas regras parecidas com o poker. A diferença em Falatro é que, além das cartas que você possui, você pode comprar coringas que podem ser utilizados como modificadores para aumentar a pontuação da sua mão, dependendo do que você possui.

Curioso com o desenvolvimento do jogo, John resolveu incluir o seu próprio coringa na lista de cartas, o fidojoker. As regras desse coringa são as seguintes: “Todas as cartas da sua mão irão possuir relações pareadas de confiança entre elas, e a pontuação da sua mão será calculada como a soma, para cada carta, do tamanho do menor ciclo de confiança que contém aquela carta multiplicado pelo índice da mesma”. Uma relação de confiança é formada por um par de cartas (A, B), em que A confia em B e B confia em A. Se uma carta não participar de nenhum ciclo de confiança, considera-se que seu tamanho é 1.

Como John tem dificuldades de raciocinar antes de jogar, e ele não é muito bom com matemática, ele pediu a sua ajuda para verificar qual é a pontuação de sua mão, utilizando o coringa fidojoker.

Input

A entrada é composta por um único caso de teste. A primeira linha da entrada contém dois inteiros N ($3 \leq N \leq 100$) e M ($2 \leq M \leq \frac{N*(N-1)}{2}$), que representam o número de cartas na mão de John e o número de relações de confiança entre as cartas, respectivamente. As próximas M linhas contém dois inteiros A e B ($1 \leq A, B \leq N$, $A \neq B$), indicando que as cartas A e B possuem uma relação de confiança entre elas. É garantido que não existem múltiplas relações entre o mesmo par de cartas.

Output

A saída deve conter um único inteiro, que representa a pontuação total da mão de John.

Examples

standard input	standard output
4 4 1 2 2 3 1 3 4 1	22
6 6 2 4 4 6 4 1 6 3 5 1 3 1	63

Note

No primeiro caso de teste, o menor ciclo formado pelas cartas é composto por 1, 2 e 3. Além disso, a carta 4 não está em nenhum ciclo. Assim, a pontuação de John é $1 * 3 + 2 * 3 + 3 * 3 + 4 * 1 = 22$.

Problem G. Gelatos do Goiás

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

Lucas é proprietário de uma nova franquia da Heladitos do Goiás, especializada na produção de gelatos com frutas típicas do cerrado. Em breve acontecerá uma competição nacional que irá eleger o melhor gelato do Brasil, e Lucas pretende participar lançando um sabor marcante: o Gelato de Pequi.

Para avaliar a qualidade de um gelato, Lucas utiliza uma fórmula definida como o produto entre a menor pontuação do subconjunto de ingredientes escolhidos e o tamanho desse subconjunto. Formalmente, sendo I o conjunto de ingredientes disponíveis e E o subconjunto de ingredientes escolhidos para o gelato g ($E \subseteq I$), a sua qualidade Q_g é dada por:

$$Q_g = \min_{x \in E} (x) \cdot |E|$$

Lucas organizou uma lista a com N valores, correspondentes à pontuação de cada ingrediente disponível. Além disso, para evitar que o odor do gelato se torne insuportável, ele definiu uma permutação caótica P , em que o i -ésimo ingrediente não pode ser utilizado em conjunto com o P_i -ésimo ingrediente.

Devido à alta demanda em sua loja, Lucas não dispõe de tempo para realizar essa escolha por conta própria. Assim, ele pede a sua ajuda para determinar, a partir da lista de ingredientes e da permutação P , qual é o maior valor possível de qualidade que pode ser obtido.

Input

A primeira linha contém um inteiro N ($2 \leq N \leq 10^5$), representando a quantidade de ingredientes disponíveis.

A segunda linha contém N inteiros a_i ($1 \leq a_i \leq 10^9$), onde cada a_i indica a pontuação do i -ésimo ingrediente disponível.

A terceira linha contém N inteiros P_i ($1 \leq P_i \leq N$), que definem a permutação caótica P . É garantido que $P_i \neq i$ para todo i , e que, para quaisquer $i \neq j$, vale $P_i \neq P_j$.

Output

Imprima uma única linha contendo um inteiro Q , representando o maior valor de qualidade possível que pode ser obtido a partir da escolha dos ingredientes, obedecendo as restrições definidas pela permutação caótica P .

Examples

standard input	standard output
5 10 5 6 7 8 2 3 4 5 1	14
4 2 3 4 5 3 4 2 1	8
5 1 4 5 6 7 5 3 2 1 4	10

Problem H. Hardcore Aura Farming

Input file: **standard input**
Output file: **standard output**
Time limit: **3 seconds**
Memory limit: **1024 megabytes**

João Carlos e Arthur são grandes streamers no mundo do Roblox. Eles decidiram se juntar para fazer uma collab!

Arthur quer fazer a melhor collab possível, então ele modelou os N jogos de Roblox que ele tem em comum com João como uma árvore: um grafo não direcionado, conexo e sem ciclos. Para estudar o impacto na comunidade, ele fará Q simulações, sendo que na i -ésima delas ele irá pedir para João jogar com seus seguidores os jogos correspondentes aos vértices no caminho* de S_i até F_i .

João tem atualmente M seguidores, cada um identificado por um inteiro diferente de 1 a M de acordo com sua habilidade no Roblox. Durante as simulações, para manter as partidas interessantes, João dediciu, para todo jogo j , que irá jogá-lo com todos os seus seguidores com identificadores de L_j a R_j . Além disso, como João é pro player, todo seguidor que jogar com ele o jogo j irá ganhar imediatamente K_j de aura (os que não jogarem ganham 0 de aura).

Assim, ao final da simulação i , todo seguidor k termina com uma aura acumulada $A_{i,k}$, que é a soma das auras que ele ganhou ao longo de todos os jogos jogados por João nessa simulação. Como parte de sua análise, Arthur gostaria de calcular o resultado W_i , que é a soma das auras acumuladas pelos seguidores com identificadores de X_i até Y_i , ou seja, $W_i = \sum_{k=X_i}^{Y_i} A_{i,k}$.

Porém, Arthur precisa fazer o seu trabalho de segurança. Ajude-o a calcular os resultados W para todas as Q simulações! Como cada W pode ser muito grande, você deve imprimir o resto de W na divisão por 998244353.

Note que as simulações são INDEPENDENTES, ou seja, a aura acumulada NÃO persiste entre simulações.

*: em uma árvore, existe apenas um caminho entre dois vértices que não passa pelo mesmo vértice mais de uma vez.

Input

A primeira linha contém dois inteiros: N ($1 \leq N \leq 5 \cdot 10^4$) e M ($1 \leq M \leq 10^9$), representando a quantidade de jogos e o número de seguidores de João Carlos.

As próximas $N - 1$ linhas contém dois inteiros distintos U e V ($1 \leq U, V \leq N$) indicando que há uma aresta entre os vértices correspondentes aos jogos U e V .

Seguem-se N linhas contendo três inteiros. A j -ésima linha contém L_j , R_j ($1 \leq L_j \leq R_j \leq M$) e K_j ($1 \leq K_j < 998244353$), os inteiros associados ao jogo j .

A próxima linha contém um inteiro Q ($1 \leq Q \leq 5 \cdot 10^4$), a quantidade de simulações.

Por fim, seguem-se Q linhas contendo quatro inteiros. **Esses inteiros estão criptografados.** Seja W_{i-1} o resultado da $(i - 1)$ -ésima simulação (se $i = 1$, $W_{i-1} = 0$). Seu programa receberá $S_i \oplus W_{i-1}$, $F_i \oplus W_{i-1}$ ($1 \leq S_i, F_i \leq N$), $X_i \oplus W_{i-1}$ e $Y_i \oplus W_{i-1}$ ($1 \leq X_i \leq Y_i \leq M$) descrevendo os parâmetros da i -ésima simulação.

Output

Escreva Q linhas, cada uma contendo um único inteiro. A i -ésima linha deve conter W_i : o resultado da i -ésima simulação.

Example

standard input	standard output
10 10	42
3 4	0
9 6	1
1 9	44
4 5	43
4 2	32
5 6	4
7 10	10
4 8	20
10 1	13
6 9 10	
4 6 8	
7 9 1	
1 3 6	
5 7 1	
3 6 1	
5 8 10	
9 9 10	
2 5 4	
6 6 7	
10	
6 2 2 6	
40 40 45 32	
3 5 9 10	
0 5 3 7	
42 46 46 38	
44 40 35 34	
38 37 35 37	
5 5 13 13	
13 12 2 2	
23 28 17 29	

Note

$\oplus$ indica a operação XOR (ou exclusivo) aplicada bit a bit. Por exemplo, 3 é 0011 em binário, e 5 é 0101.
 $3 \oplus 5 = 6$ que é 0110 em binário.

Problem I. Ivo viu o UVa

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

Ivo, notório competidor, sentiu-se saudosos do seu treinamento com o clássico UVa Online Judge e resolveu incorporá-lo em um jogo de encontrar palavras.



O jogo é simples, dada uma matriz com letras, Ivo busca as ocorrências da sequência de letras **U-V-A**, que pode ocorrer vertical, horizontal ou diagonalmente. Crie um programa que diga quantas vezes Ivo viu o UVa!

Input

A primeira linha apresenta as dimensões N e M da matriz ($3 \leq N, M \leq 100$), separadas por espaço. A seguir são dadas M linhas, cada uma com N letras.

Output

Apresente a quantidade de ocorrências da sequência de letras UVa identificadas por Ivo.

Examples

standard input	standard output
3 3 IVO VIU UVA	1
4 5 aaaa vvvv uuuu vvvv aaaa	16

Note

No segundo exemplo, cada 'v' tem um 'u' e um 'a' adjacentes verticalmente, definindo 8 ocorrências de 'UVa'. Além disso, cada um dos 4 'v' que não estão nas bordas da matriz tem uma ocorrência de 'UVa' em cada diagonal, resultando em mais 8 casos para o total de 16.

Problem J. Jogo no Grid

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

O professor Edson aproveitou a Semana Universitária para organizar uma noite de jogos com os membros da UnBalloon. Como todos são competidores, foram escolhidos jogos combinatórios imparciais!

O jogo da vez é jogado em um grid de células com N linhas e M colunas (cada uma das N linhas do grid possui M células). Inicialmente, todas as células estão vazias.

Wilson e Pedro são a próxima dupla pra jogar esse jogo, que é realizado em turnos. Wilson começa jogando, e, na sua vez, ele deve escolher uma célula vazia e escrever a letra W nela (de forma que ela não fica mais vazia). Pedro deve fazer o mesmo na sua vez, mas escrevendo a letra P . O jogador que não puder fazer a sua jogada (quando todas as células estiverem marcadas) perde.

Os dois já pensaram em estratégias bastante elaboradas para superar o seu oponente. Infelizmente, você não vai conseguir acompanhar a partida até o final, mas sabe que, por mais que um dos jogadores se esforce, a vitória com certeza será do outro. Qual dos jogadores será o vencedor?

Input

A entrada é composta de uma única linha, com dois inteiros: N e M ($1 \leq N, M \leq 100$), que representam a quantidade de linhas e colunas do grid.

Output

Imprima uma linha contendo W , se Wilson for vencer o jogo, ou P , se Pedro for vencer o jogo. É garantido que um dos jogadores sempre irá ganhar o jogo, independentemente das ações do adversário.

Examples

standard input	standard output
1 2	P
1 1	W
99 87	W

Problem K. Kronos

Input file: `standard input`
Output file: `standard output`
Time limit: 4 seconds
Memory limit: 256 megabytes

No ano 2087, a humanidade desenvolveu a tecnologia de comunicação temporal, permitindo o envio de mensagens para o passado. Você trabalha no Centro de Monitoramento Temporal e precisa processar consultas sobre essas mensagens vindas do futuro.

Você recebe N mensagens do futuro, cada uma com um timestamp t_i indicando quando a mensagem foi originalmente enviada (no futuro). Seu sistema precisa responder a Q consultas, onde cada consulta pergunta: "**Quantas mensagens foram enviadas no intervalo de tempo $[L, R]$?**"

Input

Primeira linha: dois inteiros N e Q ($1 \leq N, Q \leq 2 \cdot 10^5$)

Segunda linha: N inteiros t_i representando os timestamps das mensagens ($1 \leq t_i \leq 10^9$)

As próximas Q linhas: dois inteiros L e R para cada consulta ($1 \leq L \leq R \leq 10^9$)

Output

Para cada consulta, imprima o número de mensagens recebidas no intervalo fechado $[L, R]$.

Examples

standard input	standard output
5 3 10 1 10 7 5 1 5 6 10 2 4	2 3 0
3 3 10 15 20 1 5 10 10 15 25	0 1 2
6 5 600 500 400 300 200 100 100 100 50 350 1 1000000000 401 600 700 800	1 3 6 2 0

Problem L. Lá em cima

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

Após vários pedidos, finalmente o Campeonato Mundial de Minecraft da UnBalloon foi anunciado! Nesse campeonato, há várias categorias diferentes, sendo ostentação uma delas. Nessa categoria, o vencedor será aquele que obtiver mais diamantes.

Adrielly e Nikolle decidiram se juntar para competir nessa categoria e, para mostrar o poder que possuem, estão dispostas a coletar diamantes o suficiente para construir uma super torre e ativar um *beacon* lá em cima.

Uma torre é composta por blocos de diamante, sendo que o i -ésimo nível é composto por $(2i + 1)^2$ blocos. Ou seja, o primeiro nível é composto por 9 blocos, o segundo por 25 blocos, o terceiro por 49 blocos e assim por diante.

Adrielly e Nikolle estão decididas a construir uma torre de N níveis. No momento elas estão minerando e acabaram se esquecendo de contar quantos blocos serão necessários! Será que você pode ajudá-las?

Input

A entrada contém um único inteiro N ($1 \leq N \leq 10^{18}$), a altura da torre desejada.

Output

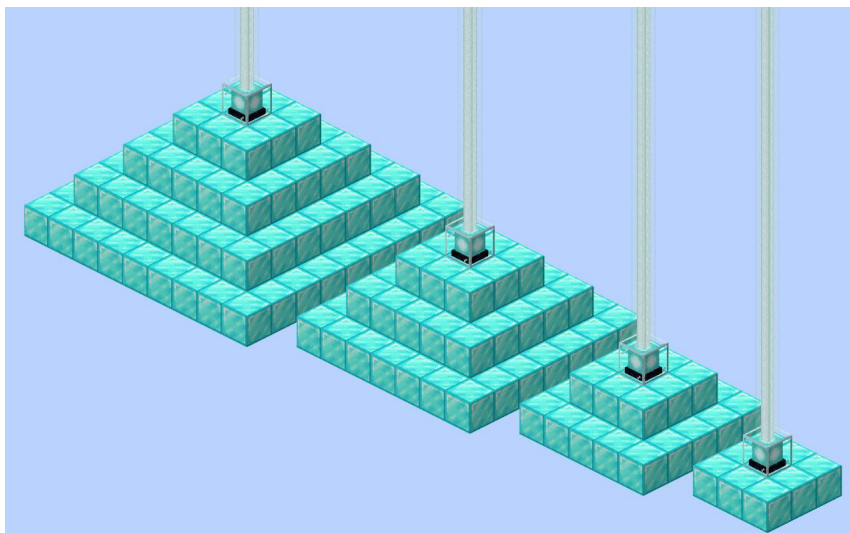
Imprima, em uma única linha, o número de blocos necessários para construir uma torre de altura N módulo $10^9 + 7$.

Examples

standard input	standard output
1	9
3	83
5	285
3271897334534761	217325831

Note

Na figura abaixo temos torres com alturas 4, 3, 2 e 1, respectivamente.



Problem M. Mapeamento Tático

Input file: **standard input**
Output file: **standard output**
Time limit: **3 seconds**
Memory limit: **512 megabytes**

O ciclo para a próxima Exatas Cup já se iniciou, e Cauê, o maestro da Selecic, já está pensando em novas táticas para buscar o título na próxima edição.

Cauê está pensando em organizar o time de forma posicional, onde cada jogador deve atuar em uma região tática restrita do campo de acordo com a sua função. Por exemplo, os atacantes devem atuar próximo ao gol adversário, os defensores no campo de defesa da equipe, e o goleiro apenas na região restrita ao gol. Porém, a ideia de função pode ser bastante complexa: dentro do espaço reservado para uma função, podem existir outras funções contidas nele!

Por exemplo, dentro do espaço dos atacantes, pode existir um espaço reservado para os que são pontas e para os que são centroavantes. E também no espaço dos pontas, pode haver um espaço para os de pé invertido e os de pé natural. Funções podem estar dentro de outras de forma arbitrária! No entanto, não pode haver duas funções que compartilhem espaço sem que uma esteja contida na outra — isso iria contra os princípios táticos de Cauê.

Para ilustrar sua ideia, Cauê usou um plano cartesiano para desenhar as regiões táticas. Cada região foi representada por um polígono convexo simples, de forma que as arestas de polígonos de regiões diferentes não se intersectam. Isso garante que, entre regiões diferentes, ou elas não tem intersecção ou uma está estritamente contida (ou seja, “dentro”) da outra.

Os outros membros da Selecic ficaram bastante confusos com o desenho de Cauê. Uma pergunta recorrente era: “Se quero começar jogando no ponto (X, Y) , quantas funções eu vou ter que desempenhar no jogo? Ou seja, o ponto (X, Y) é contido pela região de quantas funções?”. Afinal de contas, os jogadores do time não tem capacidade para exercer um número grande demais de funções na partida...

Cauê está ocupado vendo o jogo do Vasco, e por isso pediu que você ficasse no lugar dele respondendo as perguntas dos jogadores da Selecic! Ele já sofreu demais com esse time, então não custa nada fazer esse favor.

Input

A primeira linha da entrada contém dois inteiros: N e Q , representando respectivamente a quantidade de regiões desenhadas por Cauê e a quantidade de perguntas dos jogadores.

As próximas linhas contêm as descrições dos polígonos de cada uma das N regiões. A descrição do i -ésimo polígono começa com uma linha tendo o inteiro k_i : a quantidade de vértices do polígono. As próximas k_i linhas contêm cada uma dois inteiros X e Y , representando as coordenadas cartesianas de um vértice do polígono. Os vértices são dados em ordem anti-horária, e não há três vértices consecutivos colineares.

Em seguida, as próximas Q linhas contêm cada uma dois inteiros X e Y , representando as coordenadas cartesianas dos pontos associados às perguntas dos jogadores.

É garantido que as arestas dos polígonos não se intersectam. Além disso, é garantido que os pontos das perguntas não são vértices nem estão na borda de nenhum dos polígonos da entrada. Por fim, todos os pontos da entrada são distintos.

Também é garantido que $N, Q \geq 1$, $k_i \geq 3$, $-10^9 \leq X, Y \leq 10^9$ e que $(Q + \sum_{i=1}^n k_i) \leq 6 \cdot 10^5$.

Output

Imprima Q linhas: na i -ésima delas, a quantidade de regiões que contém o ponto da i -ésima pergunta.

Examples

standard input	standard output
1 2 4 -1 0 0 -1 1 0 0 1 0 0 1 1	1 0
3 3 4 1 4 3 2 9 8 2 6 3 1 0 11 9 0 8 4 2 5 3 3 6 6 3 6 3 5 2 4 1 3	3 2 1

Problem N. Névoa

Input file: **standard input**
Output file: **standard output**
Time limit: **1 second**
Memory limit: **256 megabytes**

O professor John estava em alto mar, realizando uma pesquisa sobre a fauna marinha, quando sua embarcação, que estava ancorada no ponto X , foi envolvida por uma névoa intensa que interferia tanto na visibilidade quanto nos sensores de localização do barco.

John e sua equipe aguardaram pacientemente durante dois dias, mas como não havia indicativo de que a névoa cederia e os suprimentos já estavam baixando a níveis preocupantes, ele decidiu que eles deveriam procurar por terra firme.

Para não ficarem navegando desorientados à esmo, o professor resolveu adotar a seguinte estratégia:

1. mover o barco para frente, em linha reta, por um metro, até atingir para o ponto $X + 1$;
2. em seguida, recuar o barco em 3 metros, em linha reta, até atingir o ponto $X - 2$;
3. novamente seguir em frente, até atingir o ponto $X + 4$;
4. uma vez mais recuar até o ponto $X - 8$ e assim por diante: a cada etapa ele se moverá o barco no sentido oposto da etapa anterior, até um ponto que esteja a uma distância duas vezes maior, em relação ao ponto X , do que a etapa anterior, até que encontrassem terra firme.

Foi um processo demorado e exaustivo, mas a equipe teve sucesso e ancorou em terra firme. Após ter se recuperado, o professor John resgatou o registro de seu diário de bordo, onde constava que o deslocamento total do barco foi de D metros e o ponto onde encontraram terra firme estava localizado em Y .

A partir desses dados, ele pretende determinar o ponto X em que eles estavam inicialmente ancorados. Contudo, dado o cansaço físico e mental da experiência, John pode ter se equivocado em suas notas. Ajude, portanto, a definir se os dados são ou não consistentes e, em caso afirmativo, determine o valor de X .

Input

A primeira linha da entrada contém o valor do inteiro D ($1 \leq D \leq 3 \times 10^{18}$).

A segunda linha da entrada contém o valor do inteiro Y ($-10^{18} \leq Y \leq 10^{18}$).

Output

Imprima, em uma linha, a mensagem “**Sim**” (sem as aspas), caso os dados do professor John sejam consistentes, ou a mensagem “**Nao**”, caso contrário.

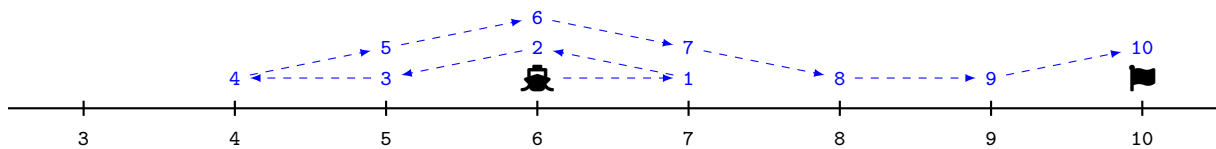
Se os dados forem consistentes imprima, na segunda, o valor de X , o ponto onde a embarcação estava inicialmente ancorada.

Examples

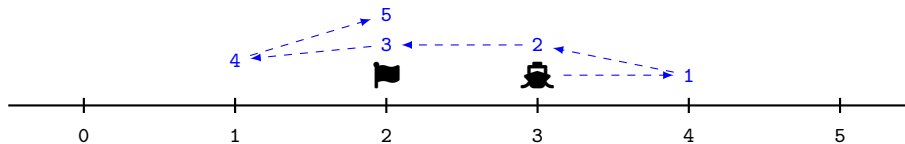
standard input	standard output
10 10	Sim 6
5 2	Nao
40 1	Sim -9
4 -3	Sim -1

Note

No primeiro caso, temos a seguinte sequência de movimentos do barco: os números em azul e as setas indicam o deslocamento total até o ponto indicado, os números em preto marcam as posições e a bandeira sinaliza o início da terra firme.



No segundo caso, para chegar ao ponto 2 com deslocamento de exatamente 5 metros, o barco deveria estar ancorado em $X = 3$ e teríamos a seguinte situação:



Note que, segundo mostra a figura, se a terra firme tivesse início no ponto $Y = 2$, o barco a teria alcançado quando o deslocamento total tivesse registrado 3 metros, portanto o registro do professor John é inconsistente.

No quarto caso, observe que tanto X quando Y podem ser negativos.